

Mathematical Formulation and Spectral Sensitivity Analysis in RMS Small-Signal Algorithms

VeraGridEngine Computational Core

February 18, 2026

Abstract

This document provides a mathematical demonstration of the enhanced algebraic algorithms implemented within the Small-Signal Stability (SSS) module for Root Mean Square (RMS) power system simulations. The theoretical proofs validate the exactness of the Schur Complement reduction, the spectral invariance of Osborne's matrix balancing, and the necessity of bi-orthonormal dual-space Krylov methods for non-Hermitian systems. Furthermore, we introduce eigenvalue sensitivity bounds to formally justify the algorithmic trade-offs of the proposed architecture relative to legacy implementations.

1 Mathematical Proof of the DAE-to-ODE Reduction

The dynamic behavior of an RMS power system is governed by a set of semi-explicit Differential-Algebraic Equations (DAEs). Let $x \in \mathbb{R}^n$ represent the dynamic state variables and $y \in \mathbb{R}^m$ represent the algebraic variables. Linearizing around an equilibrium point (x_0, y_0) yields the descriptor system:

$$\begin{bmatrix} \Delta \dot{x} \\ 0 \end{bmatrix} = \begin{bmatrix} f_x & f_y \\ g_x & g_y \end{bmatrix} \begin{bmatrix} \Delta x \\ \Delta y \end{bmatrix} \quad (1)$$

where f_x, f_y, g_x, g_y are the Jacobian sub-matrices. To transform this system into a standard Ordinary Differential Equation (ODE) of the form $\Delta \dot{x} = A\Delta x$, we analytically eliminate the algebraic constraint:

$$g_x \Delta x + g_y \Delta y = 0 \implies \Delta y = -g_y^{-1} g_x \Delta x \quad (2)$$

Substituting this relation back into the differential equation yields the Schur Complement:

$$A = f_x - f_y g_y^{-1} g_x \quad (3)$$

Evaluating the term $g_y^{-1} g_x$ requires solving the linear system $g_y Z = g_x$. By executing a single SuperLU decomposition $P_r g_y P_c = LU$, the matrix Z is computed via highly optimized forward and backward substitutions, avoiding the numerically unstable explicit inversion of g_y .

2 Eigenvalue Condition Number and Matrix Balancing

Power system state matrices are inherently stiff, causing severe numerical ill-conditioning that disrupts the convergence of the QR eigenvalue algorithm. To mitigate this, a diagonal similarity transformation D is applied to A , yielding $A_{bal} = D^{-1}AD$.

2.1 Eigenvalue Sensitivity Bound

Let λ_i be a simple eigenvalue of A with right eigenvector v_i and left eigenvector w_i . According to the Bauer-Fike theorem and perturbation theory, if A is perturbed by a matrix E (such as floating-point round-off error), the perturbed eigenvalue $\lambda_i(E)$ satisfies:

$$|\lambda_i(E) - \lambda_i| \leq \kappa(\lambda_i) \|E\| + \mathcal{O}(\|E\|^2) \quad (4)$$

where $\kappa(\lambda_i)$ is the absolute condition number of the eigenvalue, defined as:

$$\kappa(\lambda_i) = \frac{\|w_i\|_2 \|v_i\|_2}{|w_i^T v_i|} \quad (5)$$

For highly asymmetric matrices, the angle between the left and right eigenvectors can approach $\pi/2$, driving $w_i^T v_i \rightarrow 0$ and making $\kappa(\lambda_i) \rightarrow \infty$. Osborne’s balancing algorithm iteratively finds the diagonal matrix D that minimizes the Frobenius norm of A_{bal} , which consequently minimizes the product $\|w_i\|_2 \|v_i\|_2$. Thus, the sensitivity bound $\kappa(\lambda_i)$ is formally optimized without altering the theoretical spectrum.

3 Dual-Space Krylov Methods and Shift-and-Invert Spectral Transformation

For massive interconnected power grids, computing the dense matrix A is prohibitive. Instead, dominant eigenvalues are extracted using Krylov subspace methods (Arnoldi iteration). In power system stability, critical electromechanical modes lie close to the imaginary axis. To isolate these modes, the algorithm employs a Shift-and-Invert spectral transformation.

3.1 Shifted Inverse Iteration

By shifting the matrix spectrum via $B = (A - \sigma I)^{-1}$, eigenvalues λ of A that are closest to the shift σ are mapped to the largest magnitude eigenvalues of B , given by $\mu = 1/(\lambda - \sigma)$. To avoid catastrophic dense fill-in, the solver computes $(A - \sigma I)^{-1}v$ not by matrix inversion, but by solving the augmented sparse linear system at each iteration. Setting $\sigma = 0.0$ effectively targets the dominant low-frequency modes characterizing small-signal instability.

3.2 Bi-orthogonality in Non-Hermitian Subspaces

Because A is non-Hermitian ($A \neq A^*$), its right eigenvectors V are not orthogonal. To compute rigorous Participation Factors without assembling the dense state matrix, we utilize a dual-space formulation, computing left eigenvectors W from the transposed operator A^T . The formulation is formalized in Algorithm 1.

4 Disadvantages and Engineering Trade-offs

While the integration of SuperLU and dual-space Arnoldi iteration ensures commercial-grade robustness, it introduces engineering trade-offs. The explicit storage of the L and U sparse factors increases the permanent memory footprint, which can lead to memory exhaustion in pathological network topologies. Furthermore, computing both the left and right Krylov subspaces directly doubles the temporal cost of the eigenvalue extraction phase compared to legacy models.

Finally, rewriting Pythonic array broadcasts into Low Level Virtual Machine (LLVM) compatible, explicit nested loops—required to leverage Single Instruction, Multiple Data (SIMD)

Algorithm 1 Dual-Space Participation Factor Computation

Require: Abstract operators for A and A^T , number of desired modes k .

Ensure: Normalized Participation Matrix $P \in \mathbb{R}^{n \times k}$.

```
1: Compute  $k$  right eigenvectors  $V \in \mathbb{C}^{n \times k}$  such that  $AV = V\Lambda_R$ .
2: Compute  $k$  left eigenvectors  $\tilde{W} \in \mathbb{C}^{n \times k}$  such that  $A^T\tilde{W} = \tilde{W}\Lambda_L$ .
3: Sort  $\Lambda_R, V$  and  $\Lambda_L, \tilde{W}$  based on real components to align mode indices.
4: for  $i = 1$  to  $k$  do
5:    $\alpha_i \leftarrow \sum_{j=1}^n \tilde{W}_{j,i} V_{j,i}$  ▷ Dot product for bi-orthonormalization
6:   if  $|\alpha_i| > \epsilon$  then
7:      $W_{:,i} \leftarrow \tilde{W}_{:,i} / \alpha_i$  ▷ Enforce  $W_i^T V_i = 1$ 
8:   end if
9:   for  $j = 1$  to  $n$  do
10:     $\tilde{P}_{j,i} \leftarrow |W_{j,i} V_{j,i}|$  ▷ Magnitude of modal interaction
11:   end for
12:    $S_i \leftarrow \sum_{j=1}^n \tilde{P}_{j,i}$ 
13:    $P_{:,i} \leftarrow \tilde{P}_{:,i} / S_i$  ▷ Column normalization
14: end for
15: return  $P$ 
```

hardware vectorization—severely reduces codebase readability. This forces future developers to adhere to strict memory-contiguous paradigms rather than high-level matrix semantics.